DATA STRUCTURES & ALGORITHMS · C++

Recursion & Backtracking Solved Questions

Every problem from Part 16 of the Recursion & Backtracking guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

38	3	12	124
PROBLEMS	TIERS	PATTERNS	TESTS PASSING

Prepared for **Hari** · on-campus placement preparation
Companion to **Recursion-and-Backtracking-Guide.pdf** · code verified against **reference-part16-solutions.cpp**
Every solution in this document was compiled with **g++ -std=c++17 -O2 -Wall** and passes its test cases.
Generated 14 October 2026

Contents

Tier 1 — Learn the mechanics

01 LC 509 · Fibonacci Number
02 LC 344 · Reverse String
03 LC 50 · Pow(x, n)
04 LC 21 · Merge Two Sorted Lists
05 LC 779 · K-th Symbol in Grammar
06 LC 1823 · Find the Winner of the Circular Game
07 LC 78 · Subsets
08 LC 90 · Subsets II
09 LC 46 · Permutations
10 LC 77 · Combinations
11 LC 39 · Combination Sum
12 LC 17 · Letter Combinations of a Phone Number
13 LC 22 · Generate Parentheses

Tier 2 — The placement core

14 LC 40 · Combination Sum II
15 LC 216 · Combination Sum III
16 LC 47 · Permutations II
17 LC 131 · Palindrome Partitioning
18 LC 79 · Word Search
19 LC 51 · N-Queens
20 LC 52 · N-Queens II
21 LC 37 · Sudoku Solver
22 LC 93 · Restore IP Addresses
23 LC 784 · Letter Case Permutation
24 LC 912 · Sort an Array
25 LC 60 · Permutation Sequence
26 LC 241 · Different Ways to Add Parentheses
27 LC 494 · Target Sum
28 LC 1239 · Maximum Length of a Concatenated String with Unique Characters

Tier 3 — Harder, but they do appear

29 LC 212 · Word Search II
30 LC 140 · Word Break II
31 LC 698 · Partition to K Equal Sum Subsets

32 LC 301 · Remove Invalid Parentheses
33 LC 282 · Expression Add Operators
34 LC 1255 · Maximum Score Words Formed by Letters
35 LC 980 · Unique Paths III
36 LC 1079 · Letter Tile Possibilities
37 LC 526 · Beautiful Arrangement
38 LC 493 · Reverse Pairs

Tier 1 — Learn the mechanics

Thirteen problems: plain recursion first, then the backtracking template in its four basic shapes. When you can write 78, 46, 39 and 22 from a blank page, you own the template.

01 LC 509 · Fibonacci Number

EASY Pattern 4 — Recurrence from the definition, + memo

PROBLEM

Return `F(n)` where `F(0) = 0`, `F(1) = 1` and `F(n) = F(n-1) + F(n-2)`, for `0 <= n <= 30`.

APPROACH

The definition *is* the code: two base cases and one recurrence. Written naively it makes about 1.6^n calls, because `fib(n-2)` is computed once directly and again inside `fib(n-1)`.

Add a cache: the answer depends only on `n`, so store it the first time and return it every time after. That is memoization, and it is the bridge from this topic to Dynamic Programming.

Complexity O(n) time with memo (O(1.618^n) without) · O(n) space for the memo and the stack

C++ SOLUTION

```cpp
class Solution {
    int memo[31];
    int f(int n) {
        if (n <= 1) return n;                    // base cases: F(0)=0, F(1)=1
        if (memo[n] != -1) return memo[n];       // already solved once
        return memo[n] = f(n - 1) + f(n - 2);    // trust the two smaller calls
    }
public:
    int fib(int n) {
        fill(memo, memo + 31, -1);
        return f(n);
    }
};
```

WATCH OUT

Draw the recursion tree for `fib(5)` once — `fib(3)` appears twice and `fib(2)` three times. That picture is the answer to "*why is the naive version slow?*", and interviewers ask it. The follow-up "*do it in O(1) space*" is two variables in a loop.

02 LC 344 · Reverse String

EASY Pattern 1 — Reduce by one, from both ends

PROBLEM

Reverse a `vector<char>` in place.

APPROACH

Handle the outer pair, trust recursion for the inside. Swap `s[l]` and `s[r]`, then reverse `s[l+1..r-1]`. The base case is `l >= r`: zero or one character in the middle, nothing to do.

This is the shape of every "both ends" recursion — palindrome check, reversing an array, and the two-pointer loop you already know, written recursively.

Complexity O(n) time · O(n) stack space (the iterative two-pointer loop is O(1))

C++ SOLUTION

```cpp
class Solution {
    void rev(vector<char>& s, int l, int r) {
        if (l >= r) return;                    // 0 or 1 chars left: done
        swap(s[l], s[r]);                      // handle the outer pair
        rev(s, l + 1, r - 1);                  // trust recursion for the inside
    }
public:
    void reverseString(vector<char>& s) {
        rev(s, 0, (int)s.size() - 1);
    }
};
```

WATCH OUT

The base case must be `l >= r`, not `l == r`. With an even length the pointers cross without ever being equal, so `l == r` never fires and the recursion runs off the ends. Also say out loud that the iterative version uses O(1) space — the recursive one is here to practise the shape.

03 LC 50 · Pow(x, n)

MEDIUM Pattern 2 — Halve the input

PROBLEM

Compute `x` raised to `n`, where `n` is any 32-bit integer, including negative values.

APPROACH

`xⁿ = (x^(n/2))²`, times one extra `x` when `n` is odd. One recursive call on `n/2` means log₂ n levels.

For negative `n`, compute `(1/x)^(-n)`. Convert `n` to `long long` **first**: `-INT_MIN` does not fit in an `int`.

Complexity O(log n) time · O(log n) stack space

C++ SOLUTION

```cpp
class Solution {
    double power(double x, long long n) {        // n >= 0 here
        if (n == 0) return 1.0;
        double half = power(x, n / 2);           // ONE recursive call, reused
        return (n % 2 == 0) ? half * half : half * half * x;
    }
public:
    double myPow(double x, int n) {
        long long N = n;                         // -INT_MIN overflows an int
        if (N < 0) { x = 1.0 / x; N = -N; }
        return power(x, N);
    }
};
```

WATCH OUT

Two traps, both classic. **Store the half-result**: `power(x, n/2) * power(x, n/2)` makes two calls per level and is O(n), not O(log n) — correct output, wrong complexity. And `n = INT_MIN` overflows when negated as an `int`; the test `myPow(1.0, INT_MIN)` exists precisely to catch it.

04 LC 21 · Merge Two Sorted Lists

EASY Pattern 1 — Reduce by one, on a list

PROBLEM

Merge two sorted linked lists into one sorted list by splicing their nodes together.

APPROACH

Whichever head is smaller is the head of the answer. Its `next` is the merge of *everything else*, which is the same problem one node smaller — so trust recursion for it.

Base cases: if one list is empty, the answer is the other list.

Complexity O(n + m) time · O(n + m) stack space

C++ SOLUTION

```cpp
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
        if (!a) return b;                       // one list exhausted:
        if (!b) return a;                       // the rest is the other list
        if (a->val <= b->val) {
            a->next = mergeTwoLists(a->next, b); // a wins this position
            return a;
        }
        b->next = mergeTwoLists(a, b->next);     // b wins this position
        return b;
    }
};
```

WATCH OUT

The recursion depth equals the total length, so lists of 10^5 nodes can overflow the stack. Be ready to give the iterative version with a dummy node (Topic 03) and to say why you would prefer it in production. The `<=` keeps the merge stable.

05 LC 779 · K-th Symbol in Grammar

MEDIUM Pattern 4 — Recurrence from the definition

PROBLEM

Row 1 is `0`. Each next row replaces every `0` with `01` and every `1` with `10`. Return the k-th (1-indexed) symbol of row n, for n up to 30.

APPROACH

Row 30 has 2^{29} symbols, so building it is out. Instead, notice that symbol k of row n is a **child** of symbol ⌈k/2⌉ of row n − 1. The left child (odd k) copies its parent; the right child (even k) is its flip.

So ask the recursion for the parent, then copy or flip it. One call per row.

Complexity O(n) time · O(n) stack space

C++ SOLUTION

```cpp
class Solution {
public:
    int kthGrammar(int n, int k) {
        if (n == 1) return 0;                    // row 1 is just "0"
        int parent = kthGrammar(n - 1, (k + 1) / 2);
        // 0 -> "01", 1 -> "10": an odd k is the LEFT child and copies its
        // parent; an even k is the RIGHT child and is the flip of its parent
        return (k % 2 == 1) ? parent : 1 - parent;
    }
};
```

WATCH OUT

Get the parent index right: it is `(k + 1) / 2`, i.e. ceiling of k/2, for 1-indexed k. Using `k / 2` sends k = 1 to parent 0, which does not exist. The neat closed form — the answer is the parity of `popcount(k - 1)` — is worth mentioning once you have the recursion.

06 LC 1823 · Find the Winner of the Circular Game

MEDIUM Pattern 4 — Josephus recurrence

PROBLEM

n friends sit in a circle. Counting k places at a time, the k-th friend leaves; counting restarts from the next one. Return the winner (1-indexed).

APPROACH

Let J(n) be the 0-indexed survivor among n people. After the first person leaves, the remaining n − 1 form the same game, but numbered starting k places further round. So the survivor of the big game is the survivor of the small game shifted forward by k:

`J(1) = 0`, `J(n) = (J(n-1) + k) % n`.

Add 1 at the end for the 1-indexed answer.

Complexity O(n) time · O(n) stack (or O(1) as a loop)

C++ SOLUTION

```cpp
class Solution {
    // survivor's 0-indexed position among n people, counting k each time
    int josephus(int n, int k) {
        if (n == 1) return 0;                    // the last one standing
        // after the first removal the circle restarts k places later,
        // so shift the smaller answer forward by k and wrap around
        return (josephus(n - 1, k) + k) % n;
    }
public:
    int findTheWinner(int n, int k) {
        return josephus(n, k) + 1;               // the problem is 1-indexed
    }
};
```

WATCH OUT

Simulating with a `vector` and `erase` is O(n²) and is what most people write first — fine for n ≤ 500, but say that the recurrence exists. Keep the recurrence 0-indexed and convert once at the end; mixing 1-indexing into the `% n` step gives off-by-one answers.

07 LC 78 · Subsets

MEDIUM Pattern 5 — Pick / skip

PROBLEM

Return every subset of an array of distinct integers, in any order.

APPROACH

At index i you make one decision: take `nums[i]` or leave it. Two calls per level, n levels, and every leaf (i == n) is one subset — 2^n of them.

`cur` is shared by reference, so the pick branch must `pop_back()` before the skip branch runs. That push / recurse / pop is the backtracking discipline in its smallest form.

Complexity O(n · 2^n) time (2^n subsets, O(n) to copy each) · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    void dfs(const vector<int>& nums, int i) {
        if (i == (int)nums.size()) {            // decided for every element
            res.push_back(cur);
            return;
        }
        cur.push_back(nums[i]);                 // choice 1: PICK nums[i]
        dfs(nums, i + 1);
        cur.pop_back();                         // undo the choice
        dfs(nums, i + 1);                       // choice 2: SKIP nums[i]
    }
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        dfs(nums, 0);
        return res;
    }
};
```

WATCH OUT

Forgetting the `pop_back()` is the bug to watch for: the skip branch then sees an element that was supposed to be left out, and you get duplicates and missing subsets. The empty input must return `[[]]` — one subset, the empty one — not `[]`.

08 LC 90 · Subsets II

MEDIUM Pattern 8 — Duplicate skipping

PROBLEM

Return every subset of an array that may contain duplicates, without duplicate subsets.

APPROACH

Sort, so equal values are adjacent. Use the loop-from-start shape, where each call decides *which value fills the next slot*. Within one call, choosing the first `2` or the second `2` leads to the same subtree, so allow only the first:

```
if (i > start && nums[i] == nums[i-1]) continue;
```

Every node of the tree is a subset, so record `cur` on entry.

Complexity O(n · 2^n) time worst case · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    void dfs(const vector<int>& nums, int start) {
        res.push_back(cur);                      // every node is a subset
        for (int i = start; i < (int)nums.size(); i++) {
            // same value at the same depth = same subtree: skip it
            if (i > start && nums[i] == nums[i - 1]) continue;
            cur.push_back(nums[i]);
            dfs(nums, i + 1);
            cur.pop_back();
        }
    }
public:
    vector<vector<int>> subsetsWithDup(vector<int>& nums) {
        sort(nums.begin(), nums.end());          // duplicates must be adjacent
        dfs(nums, 0);
        return res;
    }
};
```

WATCH OUT

`i > start`, **not** `i > 0`. The check must only skip twins at the *same depth*. With `i > 0`, a deeper call that legitimately wants a second `2` — to build `[2,2]` — is blocked, and those subsets vanish. And without the sort the check does nothing at all.

09 LC 46 · Permutations

MEDIUM Pattern 7 — The `used` array

PROBLEM

Return every permutation of an array of distinct integers.

APPROACH

Order matters, so every slot may take **any** element not yet used — the loop starts at 0, not at a `start` index. A `used[]` array records which elements are already in `cur`.

Choose (mark used, push), explore, un-choose (unmark, pop). The goal test is `cur.size() == n`.

Complexity O(n · n!) time · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    vector<bool> used;
    void dfs(const vector<int>& nums) {
        if (cur.size() == nums.size()) {         // every slot filled
            res.push_back(cur);
            return;
        }
        for (int i = 0; i < (int)nums.size(); i++) {
            if (used[i]) continue;               // already placed earlier
            used[i] = true;  cur.push_back(nums[i]);
            dfs(nums);
            used[i] = false; cur.pop_back();     // undo BOTH changes
        }
    }
public:
    vector<vector<int>> permute(vector<int>& nums) {
        used.assign(nums.size(), false);
        dfs(nums);
        return res;
    }
};
```

WATCH OUT

Two pieces of state change on *choose*, so two must be undone on *un-choose*. Resetting `used[i]` but forgetting `pop_back()` — or the reverse — is the most common bug in this problem. The swap-based version (Part 8.2) avoids the `used` array but is harder to extend to duplicates.

10 LC 77 · Combinations

MEDIUM Pattern 6 — Loop from `start`

PROBLEM

Return every combination of k numbers chosen from 1..n.

APPROACH

Combinations are unordered, so build each one in increasing order: the next number must be bigger than the last, which is what `dfs(i + 1)` enforces. Stop when `cur` has k numbers.

Prune: if you still need `need` numbers, the loop may only go up to `n - need + 1`, otherwise there are not enough numbers left to finish.

Complexity O(k · C(n, k)) time · O(k) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    void dfs(int n, int k, int start) {
        if ((int)cur.size() == k) { res.push_back(cur); return; }
        int need = k - (int)cur.size();
        // pruning: there must be at least `need` numbers left in [i, n]
        for (int i = start; i <= n - need + 1; i++) {
            cur.push_back(i);
            dfs(n, k, i + 1);                    // i+1: each number used once
            cur.pop_back();
        }
    }
public:
    vector<vector<int>> combine(int n, int k) {
        dfs(n, k, 1);
        return res;
    }
};
```

WATCH OUT

Without the pruning bound the code is still correct, just slower — it walks into dead-end branches that can never reach size k. Interviewers often ask "*can you prune this?*" and this bound is the expected answer. Recursing on `i` instead of `i + 1` would allow `[2,2]`.

11 LC 39 · Combination Sum

MEDIUM Pattern 6 — Loop from `start`, reuse allowed

PROBLEM

Given distinct positive candidates and a target, return every combination that sums to the target. Each candidate may be used any number of times.

APPROACH

Sort the candidates. Each call picks the next number from `start` onwards, then recurses with **the same index** `i` (reuse is allowed) and a smaller `remain`.

Because the array is sorted, once `c[i] > remain` every later candidate is too big as well — `break` out of the loop.

Complexity Exponential in target / smallest candidate · O(target / min) recursion depth

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    void dfs(const vector<int>& c, int start, int remain) {
        if (remain == 0) { res.push_back(cur); return; }
        for (int i = start; i < (int)c.size(); i++) {
            if (c[i] > remain) break;            // sorted: the rest are bigger too
            cur.push_back(c[i]);
            dfs(c, i, remain - c[i]);            // i, not i+1: reuse allowed
            cur.pop_back();
        }
    }
public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        sort(candidates.begin(), candidates.end());
        dfs(candidates, 0, target);
        return res;
    }
};
```

WATCH OUT

`dfs(i)` vs `dfs(i + 1)` is the entire difference between this problem and Combination Sum II. `start` still matters even with reuse: without it you get both `[2,2,3]` and `[3,2,2]`. And `break` is only safe *because* of the sort — on unsorted input it skips valid answers.

12 LC 17 · Letter Combinations of a Phone Number

MEDIUM Pattern 6 — Choices change with depth

PROBLEM

Given a string of digits 2–9, return every letter string the number could represent on a phone keypad.

APPROACH

Depth i handles digit i, and the choices at that depth are that digit's 3 or 4 letters. Push a letter, recurse on i + 1, pop. A leaf is reached when every digit has a letter.

Complexity $O(n \cdot 4^n)$ time · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    const vector<string> keys{"", "", "abc", "def", "ghi",
                              "jkl", "mno", "pqrs", "tuv", "wxyz"};
    vector<string> res;
    string cur;
    void dfs(const string& digits, int i) {
        if (i == (int)digits.size()) { res.push_back(cur); return; }
        for (char ch : keys[digits[i] - '0']) {  // one branch per letter
            cur.push_back(ch);
            dfs(digits, i + 1);
            cur.pop_back();
        }
    }
public:
    vector<string> letterCombinations(string digits) {
        if (digits.empty()) return {};           // NOT {""}
        dfs(digits, 0);
        return res;
    }
};
```

WATCH OUT

An empty input must return `[]`, **not** `[""]`. The plain recursion reaches the base case immediately and records one empty string, so the empty case needs its own guard — it is exactly one test case, and it is the one that fails.

13 LC 22 · Generate Parentheses

MEDIUM Pattern 10 — Generation with validity counters

PROBLEM

Return every well-formed string of n pairs of parentheses.

APPROACH

Do not generate all 2^{2n} strings and filter. Only take a branch that can still lead to a valid string:

— add `(` while `open < n`; — add `)` while `close < open`.

Every string that reaches length 2n is then valid by construction.

Complexity $O(4^n / \sqrt{n})$ — the n-th Catalan number of answers, each O(n) to copy · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<string> res;
    string cur;
    void dfs(int n, int open, int close) {
        if ((int)cur.size() == 2 * n) { res.push_back(cur); return; }
        if (open < n) {                          // an opener is still available
            cur.push_back('(');
            dfs(n, open + 1, close);
            cur.pop_back();
        }
        if (close < open) {                      // only close what is open
            cur.push_back(')');
            dfs(n, open, close + 1);
            cur.pop_back();
        }
    }
public:
    vector<string> generateParenthesis(int n) {
        dfs(n, 0, 0);
        return res;
    }
};
```

WATCH OUT

The rule for `)` is `close < open`, not `close < n`. With `close < n` you generate `")("`-style invalid strings. Expect the follow-up "*how many answers are there?*" — the Catalan numbers 1, 1, 2, 5, 14, 42.

Tier 2 — The placement core

These are what actually get asked. N-Queens, Sudoku, Word Search, Permutations II, Combination Sum II, Palindrome Partitioning and merge sort are the most-asked backtracking and recursion questions in on-campus drives.

14 LC 40 · Combination Sum II

MEDIUM Pattern 8 — Duplicates, no reuse

PROBLEM

Candidates may contain duplicates and each may be used **at most once**. Return every unique combination summing to the target.

APPROACH

Combination Sum with two edits: recurse with `i + 1` (each element used once), and add the Subsets II line so two equal values are never tried for the same slot. The sorted-bound `break` still applies.

Complexity $O(n \cdot 2^n)$ worst case · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    void dfs(const vector<int>& c, int start, int remain) {
        if (remain == 0) { res.push_back(cur); return; }
        for (int i = start; i < (int)c.size(); i++) {
            if (i > start && c[i] == c[i - 1]) continue;  // duplicate branch
            if (c[i] > remain) break;                     // sorted: prune
            cur.push_back(c[i]);
            dfs(c, i + 1, remain - c[i]);                 // i+1: use once
            cur.pop_back();
        }
    }
public:
    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
        sort(candidates.begin(), candidates.end());
        dfs(candidates, 0, target);
        return res;
    }
};
```

WATCH OUT

All three lines must be right at once: `i + 1`, `i > start` (not `i > 0`), and `break`. With target 8, `[1,1,6]` is an answer — and `i > 0` would wrongly remove it.

15 LC 216 · Combination Sum III

MEDIUM Pattern 6 — Fixed size

PROBLEM

Find every combination of exactly k distinct digits from 1–9 that sums to n.

APPROACH

The candidates are just 1..9, in order, used once. Recurse with `d + 1`. When `cur` has k digits, record it only if the remaining sum is zero, and stop either way — a k-digit path can never grow.

Complexity O(C(9, k) · k) — tiny · O(k) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    void dfs(int k, int start, int remain) {
        if ((int)cur.size() == k) {
            if (remain == 0) res.push_back(cur);
            return;                              // k numbers used: stop either way
        }
        for (int d = start; d <= 9; d++) {
            if (d > remain) break;               // digits only grow from here
            cur.push_back(d);
            dfs(k, d + 1, remain - d);
            cur.pop_back();
        }
    }
public:
    vector<vector<int>> combinationSum3(int k, int n) {
        dfs(k, 1, n);
        return res;
    }
};
```

WATCH OUT

The goal test has two conditions — size **and** sum. Checking only `remain == 0` admits combinations with the wrong count; checking only the size admits the wrong sum. Returning as soon as the size is reached (whatever the sum) is also what keeps the tree small.

16 LC 47 · Permutations II

MEDIUM Pattern 8 — The `!used[i-1]` rule

PROBLEM

Return every unique permutation of an array that may contain duplicates.

APPROACH

Sort, then use the LC 46 `used[]` template with one extra condition:

```
if (i > 0 && a[i] == a[i-1] && !used[i-1]) continue;
```

Read it as a rule: **equal values are always placed in left-to-right order.** If the earlier twin is not yet used, using this one first would start an ordering the earlier twin has already explored.

Complexity O(n · n!) worst case · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<int>> res;
    vector<int> cur;
    vector<bool> used;
    void dfs(const vector<int>& a) {
        if (cur.size() == a.size()) { res.push_back(cur); return; }
        for (int i = 0; i < (int)a.size(); i++) {
            if (used[i]) continue;
            // equal values must be placed in left-to-right order; if the
            // previous twin is NOT used, this branch repeats an earlier one
            if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;
            used[i] = true;  cur.push_back(a[i]);
            dfs(a);
            used[i] = false; cur.pop_back();
        }
    }
public:
    vector<vector<int>> permuteUnique(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        used.assign(nums.size(), false);
        dfs(nums);
        return res;
    }
};
```

WATCH OUT

This is the condition people memorise without understanding, then get backwards. `!used[i-1]` and `used[i-1]` both give correct output — they enforce opposite orderings — but `!used[i-1]` prunes much earlier. If you cannot explain it, say the rule in words: twins go in order. A `set<vector<int>>` to dedupe works too, but it wastes all the duplicate work.

17 LC 131 · Palindrome Partitioning

MEDIUM Pattern 9 — Cut at every position

PROBLEM

Split a string so that every piece is a palindrome. Return every such split.

APPROACH

The choice at each step is **where the next piece ends**. From `start`, try every `end`; if `s[start..end]` is a palindrome, push it and partition the rest from `end + 1`. When `start` reaches the end of the string, `cur` is a complete partition.

> **Complexity** $O(n \cdot 2^n)$ time · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<vector<string>> res;
    vector<string> cur;
    bool isPal(const string& s, int l, int r) {
        while (l < r) if (s[l++] != s[r--]) return false;
        return true;
    }
    void dfs(const string& s, int start) {
        if (start == (int)s.size()) { res.push_back(cur); return; }
        for (int end = start; end < (int)s.size(); end++) {
            if (!isPal(s, start, end)) continue;      // this cut is illegal
            cur.push_back(s.substr(start, end - start + 1));
            dfs(s, end + 1);                          // partition the rest
            cur.pop_back();
        }
    }
public:
    vector<vector<string>> partition(string s) {
        dfs(s, 0);
        return res;
    }
};
```

WATCH OUT

Checking each piece with a two-pointer loop makes every check O(n). For long inputs precompute `isPal[i][j]` with the DP from Strings Part 6 — `isPal[i][j] = s[i]==s[j] && isPal[i+1][j-1]` — and every check becomes O(1). Mention it even if you do not write it.

18 LC 79 · Word Search

MEDIUM Pattern 12 — Mark, explore, unmark

PROBLEM

Does the word exist in the grid, built from horizontally or vertically adjacent cells, each cell used at most once?

APPROACH

Try every cell as a start. From a cell matching `word[i]`, overwrite it with `#` so this path cannot reuse it, try the four neighbours for `word[i+1]`, then **restore** the cell before returning.

`||` short-circuits, so the search stops the moment one direction succeeds.

Complexity $O(R \cdot C \cdot 3^L)$ time for a word of length L · O(L) recursion space

C++ SOLUTION

```cpp
class Solution {
    bool dfs(vector<vector<char>>& b, const string& w, int i, int r, int c) {
        if (i == (int)w.size()) return true;          // matched every letter
        if (r < 0 || c < 0 || r >= (int)b.size() || c >= (int)b[0].size())
            return false;
        if (b[r][c] != w[i]) return false;
        char saved = b[r][c];
        b[r][c] = '#';                                // mark: in the current path
        bool found = dfs(b, w, i + 1, r + 1, c) || dfs(b, w, i + 1, r - 1, c) ||
                     dfs(b, w, i + 1, r, c + 1) || dfs(b, w, i + 1, r, c - 1);
        b[r][c] = saved;                              // unmark on the way back
        return found;
    }
public:
    bool exist(vector<vector<char>>& board, string word) {
        for (int r = 0; r < (int)board.size(); r++)
            for (int c = 0; c < (int)board[0].size(); c++)
                if (dfs(board, word, 0, r, c)) return true;
        return false;
    }
};
```

WATCH OUT

Restore the cell even when the search succeeds — the solution saves the result in `found` and restores before returning, so the board is left intact. And do not use a global `visited` that is never cleared: a cell blocked for one path must be usable by another. That is the difference between backtracking DFS and graph DFS.

19 LC 51 · N-Queens

HARD Pattern 11 — Constraint placement

PROBLEM

Return every arrangement of n queens on an n × n board such that no two attack each other.

APPROACH

Place **one queen per row**, so the recursion depth is the row and rows never conflict. For O(1) attack checks keep three boolean arrays: `col[c]`, `diag[r+c]` and `anti[r-c+n-1]`. Every cell on one diagonal shares `r + c`; every cell on the other shares `r - c`.

Set all three plus the board cell on *choose*; clear all four on *un-choose*.

Complexity O(n!) upper bound, far less in practice · O(n²) for the board, O(n) recursion

C++ SOLUTION

```cpp
class Solution {
    vector<vector<string>> res;
    vector<string> board;
    vector<bool> col, diag, anti;    // diag: r+c,  anti: r-c+n-1
    void place(int r, int n) {
        if (r == n) { res.push_back(board); return; }
        for (int c = 0; c < n; c++) {
            if (col[c] || diag[r + c] || anti[r - c + n - 1]) continue;
            col[c] = diag[r + c] = anti[r - c + n - 1] = true;
            board[r][c] = 'Q';
            place(r + 1, n);                          // one queen per row
            board[r][c] = '.';
            col[c] = diag[r + c] = anti[r - c + n - 1] = false;
        }
    }
public:
    vector<vector<string>> solveNQueens(int n) {
        board.assign(n, string(n, '.'));
        col.assign(n, false);
        diag.assign(2 * n - 1, false);
        anti.assign(2 * n - 1, false);
        place(0, n);
        return res;
    }
};
```

WATCH OUT

The diagonal arrays need **2n − 1** entries, and `r - c` needs the `+ n - 1` shift to be non-negative. Get either wrong and n = 4 may pass while n = 8 writes out of bounds. Scanning the board for attacks instead of using the arrays is correct but is exactly what the interviewer is checking you avoid.

20 LC 52 · N-Queens II

HARD Pattern 11 — Bitmasks passed by value

PROBLEM

Return the number of distinct N-Queens solutions.

APPROACH

Counting only, so the function returns an `int` and adds up its children. The three occupancy sets become three integers — bit c of `cols` means column c is taken, and so on.

They are passed **by value**: each call receives its own copy with the new bits set, so there is nothing to undo.

Complexity O(n!) upper bound · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    // the three bitmasks are passed BY VALUE, so there is nothing to undo
    int place(int n, int r, int cols, int d1, int d2) {
        if (r == n) return 1;                          // a full valid board
        int count = 0;
        for (int c = 0; c < n; c++) {
            int a = r + c, b = r - c + n - 1;
            if ((cols >> c & 1) || (d1 >> a & 1) || (d2 >> b & 1)) continue;
            count += place(n, r + 1, cols | 1 << c, d1 | 1 << a, d2 | 1 << b);
        }
        return count;
    }
public:
    int totalNQueens(int n) {
        return place(n, 0, 0, 0, 0);
    }
};
```

WATCH OUT

This is the cleanest demonstration of Part 1.4: state passed by value needs no undo. It works because the state is three small integers. Use `1 << c`, not `1 << (c+1)`, and keep n ≤ 30 for an `int` mask — the diagonal masks need 2n − 1 bits.

21 LC 37 · Sudoku Solver

HARD Pattern 11 — Search, stop at the first answer

PROBLEM

Fill a 9 × 9 Sudoku board in place. Exactly one solution exists.

APPROACH

Track which digits each row, column and box holds, and list the blanks. For each blank try every legal digit: place, recurse, and on success **return true immediately**; otherwise undo. The box index is `(r / 3) * 3 + c / 3`.

Complexity O(9^blanks) bound, very fast in practice · O(81) state

C++ SOLUTION

```cpp
class Solution {
    bool row[9][10], col[9][10], box[9][10];
    vector<pair<int, int>> blanks;
    bool solve(vector<vector<char>>& b, int idx) {
        if (idx == (int)blanks.size()) return true;   // every blank filled
        auto [r, c] = blanks[idx];
        int bx = (r / 3) * 3 + c / 3;
        for (int d = 1; d <= 9; d++) {
            if (row[r][d] || col[c][d] || box[bx][d]) continue;
            row[r][d] = col[c][d] = box[bx][d] = true;
            b[r][c] = '0' + d;
            if (solve(b, idx + 1)) return true;       // stop at the FIRST solution
            row[r][d] = col[c][d] = box[bx][d] = false;
            b[r][c] = '.';
        }
        return false;                                 // no digit fits: backtrack
    }
public:
    void solveSudoku(vector<vector<char>>& board) {
        memset(row, 0, sizeof row);
        memset(col, 0, sizeof col);
        memset(box, 0, sizeof box);
        for (int r = 0; r < 9; r++)
            for (int c = 0; c < 9; c++) {
                if (board[r][c] == '.') { blanks.push_back({r, c}); continue; }
                int d = board[r][c] - '0';
                row[r][d] = col[c][d] = box[(r / 3) * 3 + c / 3][d] = true;
            }
        solve(board, 0);
    }
};
```

WATCH OUT

`return true` must come **before** the undo lines — undoing after a success erases the solution. Choosing the blank with the fewest legal digits next is the standard speed-up.

22 LC 93 · Restore IP Addresses

MEDIUM Pattern 9 — Cut, with count pruning

PROBLEM

Insert three dots into a digit string to form every valid IPv4 address — four parts, each 0–255, no leading zeros.

APPROACH

Partition the string into exactly four pieces of length 1–3. Each piece must be ≤ 255 and must not start with `0` unless it *is* `0`.

Prune by length at the top of every call: with `slots` pieces still to place, the remaining characters must number between `slots` and `3 × slots`.

Complexity $O(3^4) = O(1)$ — at most 81 leaves · O(1) space

C++ SOLUTION

```cpp
class Solution {
    vector<string> res;
    vector<string> parts;
    void dfs(const string& s, int start) {
        int left = (int)s.size() - start, slots = 4 - (int)parts.size();
        if (left < slots || left > 3 * slots) return;  // cannot fit: prune
        if (slots == 0) {
            res.push_back(parts[0] + "." + parts[1] + "." + parts[2] + "." + parts[3]);
            return;
        }
        for (int len = 1; len <= 3 && start + len <= (int)s.size(); len++) {
            string seg = s.substr(start, len);
            if (len > 1 && seg[0] == '0') break;       // no leading zeros
            if (stoi(seg) > 255) break;
            parts.push_back(seg);
            dfs(s, start + len);
            parts.pop_back();
        }
    }
public:
    vector<string> restoreIpAddresses(string s) {
        dfs(s, 0);
        return res;
    }
};
```

WATCH OUT

The leading-zero rule: `"0"` is fine, `"01"` is not. Check it before `stoi`, and `break` rather than `continue` — a longer piece starting with `0` is also invalid. The length pruning is what keeps long inputs from wasting time: a 13-digit string has no answers and should return at once.

23 LC 784 · Letter Case Permutation

MEDIUM Pattern 10 — Branch only where the rule allows

PROBLEM

Return every string formed by changing the case of any letters in `s`. Digits stay as they are.

APPROACH

Walk the string. Every character gives the branch *keep it*. A letter also gives the branch *flip its case*. So digits have one child and letters have two.

`s[i] ^= 32` flips the case of an ASCII letter, and applying it again restores it.

Complexity O(n · 2^l) for l letters · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<string> res;
    void dfs(string& s, int i) {
        if (i == (int)s.size()) { res.push_back(s); return; }
        dfs(s, i + 1);                              // branch 1: keep s[i] as is
        if (isalpha(s[i])) {                        // digits have ONE branch
            s[i] ^= 32;                             // flip case: 'a' <-> 'A'
            dfs(s, i + 1);                          // branch 2: flipped
            s[i] ^= 32;                             // undo
        }
    }
public:
    vector<string> letterCasePermutation(string s) {
        dfs(s, 0);
        return res;
    }
};
```

WATCH OUT

Do not branch on digits — that produces each string twice. And the flip must be undone before returning, because `s` is shared by reference. `^ 32` works because `'a' - 'A' == 32`; `toupper` / `tolower` are the clearer alternative if you prefer.

24 LC 912 · Sort an Array

MEDIUM Pattern 3 — Divide and conquer (merge sort)

PROBLEM

Sort an array in O(n log n) without using a built-in sort.

APPROACH

Merge sort. Split in the middle, sort each half recursively — trust it — then merge the two sorted halves with two pointers into a buffer and copy back.

Allocate the buffer **once**, outside the recursion, and reuse it.

> **Complexity** O(n log n) time always · O(n) buffer + O(log n) recursion

C++ SOLUTION

```cpp
class Solution {
    vector<int> tmp;
    void mergeSort(vector<int>& a, int lo, int hi) {   // sorts a[lo..hi]
        if (lo >= hi) return;                          // 0 or 1 element
        int mid = lo + (hi - lo) / 2;
        mergeSort(a, lo, mid);                         // trust: left half sorted
        mergeSort(a, mid + 1, hi);                     // trust: right half sorted
        int i = lo, j = mid + 1, k = lo;               // now merge the two
        while (i <= mid && j <= hi) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
        while (i <= mid) tmp[k++] = a[i++];
        while (j <= hi)  tmp[k++] = a[j++];
        for (k = lo; k <= hi; k++) a[k] = tmp[k];
    }
public:
    vector<int> sortArray(vector<int>& nums) {
        tmp.resize(nums.size());                       // one buffer, reused
        mergeSort(nums, 0, (int)nums.size() - 1);
        return nums;
    }
};
```

WATCH OUT

Allocating a fresh `vector` inside every merge call is correct but slow enough to TLE on this problem. Compute `mid` as `lo + (hi - lo) / 2`. Use `<=` in the merge comparison to keep the sort **stable** — the written-round follow-up is "*is merge sort stable, and is quick sort?*" (yes, no).

25 LC 60 · Permutation Sequence

HARD Pattern 7 — Skip whole blocks of permutations

PROBLEM

Return the k-th permutation (1-indexed, in lexicographic order) of 1..n, for n ≤ 9.

APPROACH

Do not generate permutations. There are (n − 1)! permutations starting with each digit, so the first digit is the one whose block contains k. Pick it, remove it, reduce k to its offset within the block, and recurse on the remaining digits.

Switch k to 0-indexed once at the start and the arithmetic is just `/` and `%`.

Complexity O(n²) time (erasing from a vector) · O(n) space

C++ SOLUTION

```cpp
class Solution {
    // pick the first digit, then recurse on the remaining digits with k shrunk
    string build(vector<int>& digits, int k, const vector<int>& fact) {
        int n = digits.size();
        if (n == 0) return "";
        int block = fact[n - 1];                    // perms per leading digit
        int idx = k / block;                        // which block k falls in
        int d = digits[idx];
        digits.erase(digits.begin() + idx);
        return char('0' + d) + build(digits, k % block, fact);
    }
public:
    string getPermutation(int n, int k) {
        vector<int> fact(n + 1, 1), digits;
        for (int i = 1; i <= n; i++) { fact[i] = fact[i - 1] * i; digits.push_back(i); }
        return build(digits, k - 1, fact);          // k-1: switch to 0-indexed
    }
};
```

WATCH OUT

The off-by-one: with 1-indexed k, the block is `(k - 1) / (n-1)!`, not `k / (n-1)!`. Convert to 0-indexed at the entry point and never think about it again. Generating with `next_permutation` k times works for small k, but is O(k · n) and too slow for k near 9!.

26 LC 241 · Different Ways to Add Parentheses

MEDIUM Pattern 3 — Split on the last operator, + memo

PROBLEM

Return every value an arithmetic expression of `+`, `-`, `*` can take under all possible parenthesisations.

APPROACH

Choose each operator as the **last** one evaluated. Everything to its left becomes one sub-expression and everything to its right another; solve both recursively and combine every left value with every right value.

If a range contains no operator, it is a number. Memoise on the range `(lo, hi)`, because the same sub-expression is reached from many splits.

Complexity Catalan-many results; memo makes each range computed once · $O(n^2)$ ranges

C++ SOLUTION

```cpp
class Solution {
    map<pair<int, int>, vector<int>> memo;
    vector<int> solve(const string& e, int lo, int hi) {   // e[lo..hi]
        if (memo.count({lo, hi})) return memo[{lo, hi}];
        vector<int> res;
        for (int i = lo; i <= hi; i++) {
            char op = e[i];
            if (op != '+' && op != '-' && op != '*') continue;
            // split at this operator: it is the LAST one evaluated
            vector<int> L = solve(e, lo, i - 1), R = solve(e, i + 1, hi);
            for (int a : L)
                for (int b : R)
                    res.push_back(op == '+' ? a + b : op == '-' ? a - b : a * b);
        }
        if (res.empty()) res.push_back(stoi(e.substr(lo, hi - lo + 1)));  // a number
        return memo[{lo, hi}] = res;
    }
public:
    vector<int> diffWaysToCompute(string expression) {
        return solve(expression, 0, (int)expression.size() - 1);
    }
};
```

WATCH OUT

Results may contain repeated values (`23-45` gives `-10` twice) — that is correct, do not dedupe. Numbers can have more than one digit, which is why the base case parses the whole range with `stoi` rather than reading one character.

27 LC 494 · Target Sum

MEDIUM Pattern 5 — Pick a sign, + memo

PROBLEM

Put `+` or `-` in front of every number. Count the assignments whose total equals the target.

APPROACH

Pick / skip with a twist: each element is always used, and the two branches are its two signs. The answer depends only on `(i, sum)`, and many sign choices reach the same pair, so cache it.

`sum` can be negative, so store at `memo[i][sum + total]` where `total` is the sum of all the numbers.

Complexity O(n · total) time and space with memo (O(2^n) without)

C++ SOLUTION

```cpp
class Solution {
    int total;
    vector<vector<int>> memo;          // memo[i][sum + total]
    int ways(const vector<int>& a, int i, int sum, int target) {
        if (i == (int)a.size()) return sum == target ? 1 : 0;
        int& m = memo[i][sum + total];
        if (m != -1) return m;
        return m = ways(a, i + 1, sum + a[i], target)    // put '+'
                 + ways(a, i + 1, sum - a[i], target);   // put '-'
    }
public:
    int findTargetSumWays(vector<int>& nums, int target) {
        total = accumulate(nums.begin(), nums.end(), 0);
        if (abs(target) > total) return 0;
        memo.assign(nums.size(), vector<int>(2 * total + 1, -1));
        return ways(nums, 0, 0, target);
    }
};
```

WATCH OUT

The index shift `sum + total` is mandatory — a negative index is an out-of-bounds write. Return 0 early if `|target| > total`, which also guarantees the shifted index stays in range. Zeros are tricky: `[0,0,1]`, target 1 has 4 answers, because +0 and −0 are different assignments.

28 LC 1239 · Maximum Length of a Concatenated String with Unique Characters

MEDIUM Pattern 5 — Pick / skip with bitmasks

PROBLEM

Choose a subsequence of words whose concatenation has all distinct letters. Return the maximum possible length.

APPROACH

Turn each word into a 26-bit mask, dropping words that repeat a letter internally. Then pick / skip: skip word i, or take it if `(used & mask) == 0`. The return value is the best length reachable from here, so each call returns `max(skip, take)`.

Complexity $O(2^n)$ time for $n \le 16$ words · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<int> masks;                // one bitmask per usable word
    int dfs(int i, int used) {
        if (i == (int)masks.size()) return __builtin_popcount(used);
        int best = dfs(i + 1, used);                    // skip word i
        if ((used & masks[i]) == 0)                     // no shared letter
            best = max(best, dfs(i + 1, used | masks[i]));   // take word i
        return best;
    }
public:
    int maxLength(vector<string>& arr) {
        for (const string& w : arr) {
            int m = 0;
            bool ok = true;
            for (char c : w) {
                if (m >> (c - 'a') & 1) { ok = false; break; }  // repeats itself
                m |= 1 << (c - 'a');
            }
            if (ok) masks.push_back(m);
        }
        return dfs(0, 0);
    }
};
```

WATCH OUT

Filter out words like `"aa"` **before** the search: their mask has fewer bits than their length, so the overlap test cannot detect the repeat and the length would be overcounted. The final answer is `popcount(used)`, which is why the masks make this so short.

Tier 3 — Harder, but they do appear

The same template with heavier pruning, shared resources, a trie, or a memo. Attempt these once Tier 2 is comfortable.

29 LC 212 · Word Search II

HARD Pattern 12 — Grid DFS walking a trie

PROBLEM

Return every word from a list that can be traced in the grid (adjacent cells, each used once per word).

APPROACH

Running LC 79 per word repeats shared prefixes. Put every word in a **trie** and walk board and trie together: no trie edge for this letter means no word continues here, so stop. A node that ends a word stores the word itself.

Complexity $O(R \cdot C \cdot 4 \cdot 3^{L-1})$ worst case for max word length L · O(total letters) for the trie

C++ SOLUTION

```
class Solution {
    struct Node {
        Node* next[26] = {nullptr};
        string word;                          // non-empty at the end of a word
    };
    vector<string> res;
    void dfs(vector<vector<char>>& b, int r, int c, Node* node) {
        if (r < 0 || c < 0 || r >= (int)b.size() || c >= (int)b[0].size()) return;
        char ch = b[r][c];
        if (ch == '#' || !node->next[ch - 'a']) return;  // used, or no word goes here
        node = node->next[ch - 'a'];
        if (!node->word.empty()) {
            res.push_back(node->word);
            node->word.clear();               // report each word only once
        }
        b[r][c] = '#';
        dfs(b, r + 1, c, node); dfs(b, r - 1, c, node);
        dfs(b, r, c + 1, node); dfs(b, r, c - 1, node);
        b[r][c] = ch;
    }
public:
    vector<string> findWords(vector<vector<char>>& board, vector<string>& words) {
        Node* root = new Node();
        for (const string& w : words) {       // all words share ONE trie
            Node* cur = root;
            for (char ch : w) {
                if (!cur->next[ch - 'a']) cur->next[ch - 'a'] = new Node();
                cur = cur->next[ch - 'a'];
            }
            cur->word = w;
        }
        for (int r = 0; r < (int)board.size(); r++)
```

```
            for (int c = 0; c < (int)board[0].size(); c++)
                dfs(board, r, c, root);
        return res;
    }
};
```

WATCH OUT

Report each word once — clear `node->word` after recording it. Follow-up: prune trie nodes whose subtree has no words left.

30 LC 140 · Word Break II

HARD Pattern 9 — Cut at every position, + memo

PROBLEM

Insert spaces into `s` to make every sentence whose words are all in the dictionary. Return all of them.

APPROACH

From `start`, try every end position; if `s[start..end)` is a word, combine it with every sentence for the rest of the string.

The sentences for a suffix depend only on `start`, so memoise `start → list of sentences`. The empty suffix has exactly one sentence: the empty one.

Complexity Output-sensitive — can be exponential in the number of answers · O(n) memo entries

C++ SOLUTION

```cpp
class Solution {
    unordered_set<string> dict;
    unordered_map<int, vector<string>> memo;    // start -> sentences for s[start..]
    vector<string> dfs(const string& s, int start) {
        if (memo.count(start)) return memo[start];
        vector<string> res;
        if (start == (int)s.size()) { res.push_back(""); return res; }
        for (int end = start + 1; end <= (int)s.size(); end++) {
            string word = s.substr(start, end - start);
            if (!dict.count(word)) continue;
            for (const string& rest : dfs(s, end))
                res.push_back(rest.empty() ? word : word + " " + rest);
        }
        return memo[start] = res;
    }
public:
    vector<string> wordBreak(string s, vector<string>& wordDict) {
        dict.insert(wordDict.begin(), wordDict.end());
        return dfs(s, 0);
    }
};
```

WATCH OUT

The base case must return `[""]` (one empty sentence), not `[]`. With `[]`, nothing ever combines and every answer is lost. Joining needs care too: do not add a trailing space when the rest is empty.

31 LC 698 · Partition to K Equal Sum Subsets

MEDIUM Pattern 11 — Buckets with symmetry pruning

PROBLEM

Can the array be split into k non-empty subsets with equal sums?

APPROACH

Each bucket must reach `sum / k`. Place the numbers one at a time, trying each bucket that still has room.

Three prunes make it fast: sort **descending** so large numbers fail early; skip a bucket whose fill equals the previous bucket's (the two are interchangeable); and if placing into an *empty* bucket failed, stop — every other empty bucket would fail the same way.

> **Complexity** $O(k^n)$ bound, fast in practice with pruning · O(n) recursion

C++ SOLUTION

```cpp
class Solution {
    vector<int> bucket;
    int side;
    bool dfs(const vector<int>& a, int i) {
        if (i == (int)a.size()) return true;              // every number placed
        for (int b = 0; b < (int)bucket.size(); b++) {
            if (bucket[b] + a[i] > side) continue;
            // two buckets with the same fill are interchangeable: try one
            if (b > 0 && bucket[b] == bucket[b - 1]) continue;
            bucket[b] += a[i];
            if (dfs(a, i + 1)) return true;
            bucket[b] -= a[i];
            if (bucket[b] == 0) break;                    // empty bucket failed:
        }                                                 // every empty one will
        return false;
    }
public:
    bool canPartitionKSubsets(vector<int>& nums, int k) {
        int sum = accumulate(nums.begin(), nums.end(), 0);
        if (sum % k) return false;
        side = sum / k;
        sort(nums.rbegin(), nums.rend());                 // big numbers fail fast
        if (nums[0] > side) return false;
        bucket.assign(k, 0);
        return dfs(nums, 0);
    }
};
```

WATCH OUT

Without the symmetry prunes this times out — k empty buckets produce k identical subtrees for the first number. Check `sum % k` and `max > sum / k` before searching. The bitmask-DP solution is the follow-up if the interviewer wants a guaranteed bound.

32 LC 301 · Remove Invalid Parentheses

HARD Pattern 10 — Deletion budgets

PROBLEM

Remove the minimum number of brackets to make the string valid. Return every distinct result.

APPROACH

First compute *how many* of each bracket must go: scan once; a `)` with no open partner must be deleted, and every `(` still open at the end must be deleted. Those counts `l` and `r` are the budgets.

Then backtrack character by character: delete a bracket if its budget allows, or keep it if the running balance stays non-negative. Accept at the end only if both budgets are spent and the balance is zero.

Complexity $O(2^n)$ worst case · O(n) recursion space

C++ SOLUTION

```cpp
class Solution {
    unordered_set<string> res;
    string cur;
    // l, r: how many '(' and ')' we STILL have to delete; open: balance so far
    void dfs(const string& s, int i, int l, int r, int open) {
        if (i == (int)s.size()) {
            if (l == 0 && r == 0 && open == 0) res.insert(cur);
            return;
        }
        char c = s[i];
        if (c == '(' && l > 0) dfs(s, i + 1, l - 1, r, open);    // delete it
        if (c == ')' && r > 0) dfs(s, i + 1, l, r - 1, open);    // delete it
        cur.push_back(c);                                        // keep it
        if (c != '(' && c != ')')  dfs(s, i + 1, l, r, open);
        else if (c == '(')         dfs(s, i + 1, l, r, open + 1);
        else if (open > 0)         dfs(s, i + 1, l, r, open - 1);
        cur.pop_back();
    }
public:
    vector<string> removeInvalidParentheses(string s) {
        int l = 0, r = 0;                    // the MINIMUM deletions, counted first
        for (char c : s) {
            if (c == '(') l++;
            else if (c == ')') { if (l > 0) l--; else r++; }
        }
        dfs(s, 0, l, r, 0);
        return vector<string>(res.begin(), res.end());
    }
};
```

WATCH OUT

Deleting either of two adjacent equal brackets gives the same string, so results must go into a **set**. The budgets are what guarantee *minimum* removals — without them you would have to BFS by number of removals, which is the other standard solution.

33 LC 282 · Expression Add Operators

HARD Pattern 9 — Cut, plus the `last` trick

PROBLEM

Insert `+`, `-` or `*` between the digits so the expression evaluates to the target. Return every such expression.

APPROACH

Cut the string into numbers, and before each number after the first choose an operator. Carry `val` (the value so far) and `last` (the last term added).

`+` and `-` are simple. For `*`, *the previous term must be multiplied, not added: undo it and add the product,* `val - last + last * x`, and the new `last` is `last * x`.

Complexity $O(4^n \cdot n)$ time · $O(n)$ recursion space

C++ SOLUTION

```cpp
class Solution {
    vector<string> res;
    // val: value of expr so far; last: the last multiplicative term in it
    void dfs(const string& num, int target, int pos, string expr,
             long long val, long long last) {
        if (pos == (int)num.size()) {
            if (val == target) res.push_back(expr);
            return;
        }
        for (int len = 1; pos + len <= (int)num.size(); len++) {
            if (len > 1 && num[pos] == '0') break;           // no "05"
            string s = num.substr(pos, len);
            long long x = stoll(s);
            if (pos == 0) { dfs(num, target, len, s, x, x); continue; }
            dfs(num, target, pos + len, expr + "+" + s, val + x,  x);
            dfs(num, target, pos + len, expr + "-" + s, val - x, -x);
            // '*' binds tighter: take back `last`, then add last * x
            dfs(num, target, pos + len, expr + "*" + s,
                val - last + last * x, last * x);
        }
    }
public:
    vector<string> addOperators(string num, int target) {
        dfs(num, target, 0, "", 0, 0);
        return res;
    }
};
```

WATCH OUT

Three separate traps. **Leading zeros**: `"05"` is not a number — `break` once a piece starts with `0` and is longer than one digit. **Overflow**: use `long long` for `val`, `last` and `x`. **The first number** takes no operator, so it needs its own branch.

34 LC 1255 · Maximum Score Words Formed by Letters

HARD Pattern 5 — Pick / skip with a shared resource

PROBLEM

Each word may be used once; each letter in the pool may be used once. Return the maximum total score of a set of words you can form.

APPROACH

Pick / skip over the words, with the letter counts as shared state. To take a word, decrement its letters; if any count goes negative, the word cannot be afforded. Recurse, then **restore the counts whether or not the word was affordable**.

Complexity $O(2^n \cdot L)$ for $n \le 14$ words of total length $L \cdot O(n)$ recursion

C++ SOLUTION

```cpp
class Solution {
    int cnt[26] = {0};
    int dfs(const vector<string>& w, const vector<int>& score, int i) {
        if (i == (int)w.size()) return 0;
        int best = dfs(w, score, i + 1);                // skip word i
        int gain = 0;
        bool ok = true;
        for (char c : w[i]) {                           // try to take word i
            if (--cnt[c - 'a'] < 0) ok = false;
            gain += score[c - 'a'];
        }
        if (ok) best = max(best, gain + dfs(w, score, i + 1));
        for (char c : w[i]) cnt[c - 'a']++;             // undo, ok or not
        return best;
    }
public:
    int maxScoreWords(vector<string>& words, vector<char>& letters, vector<int>& score) {
        for (char c : letters) cnt[c - 'a']++;
        return dfs(words, score, 0);
    }
};
```

WATCH OUT

The undo loop must run even when `ok` is false — the decrements already happened. Restoring only on the success path corrupts the counts for every later branch. Doing the decrement and the check in one loop keeps the do and undo exactly symmetric.

35 LC 980 · Unique Paths III

HARD Pattern 12 — Visit every cell exactly once

PROBLEM

Count the paths from start to end that walk over every non-obstacle cell exactly once.

APPROACH

Count the empty cells first. DFS from the start, blocking each cell as you step on it and unblocking it on the way back, carrying the number of moves still required. At the end cell, the path counts only if that number is zero.

Complexity O(3^(R·C)) bound, grid ≤ 20 cells · O(R · C) recursion

C++ SOLUTION

```cpp
class Solution {
    int R, C, toVisit;
    int dfs(vector<vector<int>>& g, int r, int c, int left) {
        if (r < 0 || c < 0 || r >= R || c >= C || g[r][c] == -1) return 0;
        if (g[r][c] == 2) return left == 0 ? 1 : 0;      // end: was EVERY cell used?
        g[r][c] = -1;                                    // block it for this path
        int paths = dfs(g, r + 1, c, left - 1) + dfs(g, r - 1, c, left - 1)
                  + dfs(g, r, c + 1, left - 1) + dfs(g, r, c - 1, left - 1);
        g[r][c] = 0;                                     // unblock
        return paths;
    }
public:
    int uniquePathsIII(vector<vector<int>>& grid) {
        R = grid.size(); C = grid[0].size();
        int sr = 0, sc = 0, empty = 0;
        for (int r = 0; r < R; r++)
            for (int c = 0; c < C; c++) {
                if (grid[r][c] == 0) empty++;
                if (grid[r][c] == 1) { sr = r; sc = c; }
            }
        // moves needed: every empty cell, plus the final step onto the end
        return dfs(grid, sr, sc, empty + 1);
    }
};
```

WATCH OUT

Off-by-one on the move count: the path must cover every empty cell **and** step onto the end, so the budget is `empty + 1`. Reaching the end with moves left over is a dead end, not a success — return 0 there instead of continuing past it.

36 LC 1079 · Letter Tile Possibilities

MEDIUM Pattern 7 — Branch on distinct counts

PROBLEM

Count every non-empty sequence of letters that can be made from the tiles (each tile used at most once).

APPROACH

Permutations with duplicates, of every length. Branch on **distinct letters** with a count array instead of on tile positions: take one of letter c, count this sequence plus all its extensions, put it back.

Because each branch is a distinct letter, equal tiles can never produce the same sequence twice.

Complexity Bounded by the number of answers, ≤ 7 tiles · O(n) recursion

C++ SOLUTION

```cpp
class Solution {
    int cnt[26] = {0};
    int dfs() {
        int total = 0;
        for (int c = 0; c < 26; c++) {       // branch on DISTINCT letters only,
            if (cnt[c] == 0) continue;       // so duplicates never double-count
            cnt[c]--;
            total += 1 + dfs();              // this sequence + all its extensions
            cnt[c]++;
        }
        return total;
    }
public:
    int numTilePossibilities(string tiles) {
        for (char ch : tiles) cnt[ch - 'A']++;
        return dfs();
    }
};
```

WATCH OUT

Branching on tile positions (`used[]`) counts `"AB"` twice when there are two `A`s. Branching on the count array removes the need for any duplicate-skip rule. The `1 +` counts the current sequence itself — leave it out and you only count the empty one.

37 LC 526 · Beautiful Arrangement

MEDIUM Pattern 7 — Fill the most constrained position first

PROBLEM

Count permutations of 1..n where, at every position i, either the number divides i or i divides the number.

APPROACH

Ordinary `used[]` permutations, rejecting any number that fails the divisibility test at the current position.

Fill positions from **n down to 1**. Large positions have few compatible numbers, so the tree is cut hard near the root, where a cut removes the most work.

Complexity Far below O(n!) for n ≤ 15 because of pruning · O(n) recursion

C++ SOLUTION

```cpp
class Solution {
    vector<bool> used;
    int dfs(int n, int pos) {
        if (pos == 0) return 1;              // filled positions n..1
        int count = 0;
        for (int v = 1; v <= n; v++) {
            if (used[v] || (v % pos != 0 && pos % v != 0)) continue;
            used[v] = true;
            count += dfs(n, pos - 1);
            used[v] = false;
        }
        return count;
    }
public:
    int countArrangement(int n) {
        used.assign(n + 1, false);
        // fill from position n DOWN: large positions have few divisors,
        // so the tree is pruned near the root where pruning is worth most
        return dfs(n, n);
    }
};
```

WATCH OUT

Both directions pass, but on n = 15 filling from position 1 explores many more nodes — position 1 accepts every number. This is the clearest example of *prune near the root*: the same check, applied in a better order.

38 LC 493 · Reverse Pairs

HARD Pattern 3 — Count during the merge

PROBLEM

Count pairs i < j with `nums[i] > 2 * nums[j]`.

APPROACH

Merge sort. Pairs inside each half are counted recursively. For pairs that cross the middle, both halves are already sorted, so one forward sweep counts them: for each `i` in the left half, advance `j` in the right half while `a[i] > 2 * a[j]`, and add how far `j` has moved.

Then do the ordinary merge.

Complexity O(n log n) time · O(n) buffer

C++ SOLUTION

```cpp
class Solution {
    vector<int> tmp;
    long long sortCount(vector<int>& a, int lo, int hi) {
        if (lo >= hi) return 0;
        int mid = lo + (hi - lo) / 2;
        long long cnt = sortCount(a, lo, mid) + sortCount(a, mid + 1, hi);
        // both halves are sorted now: count cross pairs with one sweep
        int j = mid + 1;
        for (int i = lo; i <= mid; i++) {
            while (j <= hi && (long long)a[i] > 2LL * a[j]) j++;
            cnt += j - (mid + 1);
        }
        // then the ordinary merge
        int i = lo, k = lo; j = mid + 1;
        while (i <= mid && j <= hi) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
        while (i <= mid) tmp[k++] = a[i++];
        while (j <= hi)  tmp[k++] = a[j++];
        for (k = lo; k <= hi; k++) a[k] = tmp[k];
        return cnt;
    }
public:
    int reversePairs(vector<int>& nums) {
        tmp.resize(nums.size());
        return (int)sortCount(nums, 0, (int)nums.size() - 1);
    }
};
```

WATCH OUT

`2 * a[j]` overflows `int` when `a[j]` is near 2^{31} — cast to `long long` first, as the test with `INT_MAX` checks. Count **before** merging and with a separate sweep: the condition `a[i] > 2·a[j]` is not the merge order, so you cannot count inside the merge loop as you can for plain inversions.